Anders: Modal Homotopy Type System for Computable Modal Homotopy Type Theory and Differential Geometry

Maksym Sokhatskyi
Groupoid Infinity

Abstract

Derivative work of **cubicaltt** with **cohesivett** and **stt** features.

2025 ACM Subject Classification Theory of computation → Lambda calculus

Keywords and phrases Homotopy Type Theory, Differential Geometry, Cohesive Type Theory, Simplicial Type Theory, Modal Type Theory, Foundations of Mathematics

Contents

1 Introduction

Anders is a Modal HoTT proof assistant based on: classical MLTT-80 [6] with 0, 1, 2, W types; CCHM [11] in CHM [2] flavour as cubical type system with hcomp/transp operations; HTS [8] strict equality on pretypes; infinitisemal [1] modality primitives for differential geometry purposes; Disc and Coequalizer primitives. We tend not to touch general recursive higher inductive schemes, instead we will try to express as much HIT as possible through Hub Spoke Disc, Coequalizer, and W primitives built into type checker core in the style of HoTT/Coq homotopy library and Three-HIT theorem [19].

The HTS language proposed by Voevodsky exposes two different presheaf models of type theory: the inner one is homotopy type system presheaf that models HoTT and the outer one is traditional Martin-Löf type system presheaf that models set theory with UIP. The motivation behind this doubling is to have an ability to express semisemplicial types. Theoretical work on merging inner and outer languages was continued in 2LTT [9].

Installation. While we are on our road to Lean-like tactic language, currently we are at the stage of regular cubical HTS type checker with CHM-style primitives. You may try it from Github sources: groupoid/anders[1] or install through OPAM package manager. Main commands are **check** (to check a program) and **repl** (to enter the proof shell).

```
$ opam install anders
```

Anders is fast, idiomatic and educational. We carefully draw the favourite Lean-compatible syntax to fit 300 LOC in Menhir. The CHM kernel is 2K LOC. Whole Anders compiles under 3 seconds and checks all the base library under 1 minute [i5-12400]. Anders proof assistant as Homotopy Type System comes with its own Homotopy Library[2].

2 Syntax

The syntax resembles original syntax of the reference CCHM type checker cubicaltt, is slightly compatible with Lean syntax and contains the full set of Cubical Agda [10] primitives (except generic higher inductive schemes).

Here is given the mathematical pseudo-code notation of the language expressions that come immediately after parsing. The core syntax definition of HTS language corresponds to the type defined in OCaml module:

Further Menhir BNF notation will be used to describe the top-level language E parser.

Keywords. The words of a top-level language, file or repl, consist of keywords or identifiers. The keywords are following: `module`, `where`, `import`, `option`, `def`, `axiom`, `postulate`, `theorem`, `(`, `)`, `[`, `]`, `<`, `>`, `/`, `.1`, `.2`, Π, Σ, `,`, λ, `V`, ⋁, ⋀, `-`, `+`, `@`, `PathP`, `transp`, `hcomp`, `zero`, `one`, `Partial`, `inc`, ×, →, `:`, `:=`, ↦, `U`, `ouc`, `interval`, `inductive`, `Glue`, `glue`, `unglue`, ♭, `♭-unit`, `♭-counit`, `ind-♭`, `coequ`, ι_2, `resp`, `coequ-ind`, `disc`, `base`, `hub`, `spoke`, `disc-ind`.

Indentifiers. Identifiers support UTF-8. Indentifiers couldn't start with `:`, `-`, →. Sample identifiers: `¬-of-∨`, `1→1`, `is-?`, `=`, `$∼]!005x`, ∞, `x→Nat`.

Modules. Modules represent files with declarations. More accurate, BNF notation of module consists of imports, options and declarations.

---

[1] `https://github.com/groupoid/anders/`
[2] `https://anders.groupoid.space/library/`

$$
\begin{aligned}
\text{cosmos} &:= \mathbf{U}_j \mid \mathbf{V}_k \\
\text{var} &:= \mathbf{var}\ name \mid \mathbf{hole} \\
\text{pi} &:= \Pi\ name\ E\ E \mid \lambda\ name\ E\ E \mid E\ E \\
\text{sigma} &:= \Sigma\ name\ E\ E \mid (E,E) \mid E.1 \mid E.2 \\
0 &:= \mathbf{0} \mid \mathbf{ind}_0\ \text{E E E} \\
1 &:= \mathbf{1} \mid \star \mid \mathbf{ind}_1\ \text{E E E} \\
2 &:= \mathbf{2} \mid 0_2 \mid 1_2 \mid \mathbf{ind}_2\ \text{E E E} \\
\text{W} &:= \mathbf{W}\ \text{ident E E} \mid \mathbf{sup}\ \text{E E} \mid \mathbf{ind}_W\ \text{E E} \\
\text{id} &:= \mathbf{Id}\ E \mid \mathbf{ref}\ E \mid \mathbf{id}_J\ E \\
\text{path} &:= \mathbf{Path}\ E \mid E^i \mid E\ @\ E \\
\text{I} &:= \mathbf{I} \mid 0 \mid 1 \mid E \bigvee E \mid E \bigwedge E \mid \neg E \\
\text{part} &:= \mathbf{Partial}\ E\ E \mid [\ (E = I) \to E, \ldots\ ] \\
\text{sub} &:= \mathbf{inc}\ E \mid \mathbf{ouc}\ E \mid E\ [\ I \mapsto E\ ] \\
\text{kan} &:= \mathbf{transp}\ E\ E \mid \mathbf{hcomp}\ E \\
\text{glue} &:= \mathbf{Glue}\ E \mid \mathbf{glue}\ E \mid \mathbf{unglue}\ E\ E \\
\text{Im} &:= \mathbf{Im}\ \text{E} \mid \mathbf{Inf}\ \text{E} \mid \mathbf{Join}\ \text{E} \mid \mathbf{ind}_{Im}\ \text{E E} \\
\text{Flat} &:= \flat\ \text{E} \mid \mathbf{unit}_\flat\ \text{E} \mid \mathbf{counit}_\flat\ \text{E} \mid \mathbf{ind}_\flat\ \text{E E} \\
\text{coequ} &:= \mathbf{coequ}\ \text{E} \mid \iota_2 \mid \mathbf{resp} \mid \mathbf{ind}_{coequ} \\
\text{disc} &:= \mathbf{disc}\ \text{E} \mid \mathbf{base} \mid \mathbf{hub} \mid \mathbf{spoke} \mid \mathbf{ind}_{disc}
\end{aligned}
$$

$$
\begin{aligned}
\text{E} &:= \text{cosmos} \mid \text{var} \mid \text{MLTT} \mid \text{CCHM} \mid \text{Im} \mid \text{Flat} \mid \text{HIT} \\
\text{CCHM} &:= \text{path} \mid \text{I} \mid \text{part} \mid \text{sub} \mid \text{kan} \mid \text{glue} \\
\text{MLTT} &:= \text{pi} \mid \text{sigma} \mid \text{id} \\
\text{HIT} &:= \text{coequ} \mid \text{disc}
\end{aligned}
$$

```
menhir
   start <Module.file> file
   start <Module.command> repl
   repl : COLON IDENT exp1 EOF | COLON IDENT EOF | exp0 EOF | EOF
   file : MODULE IDENT WHERE line* EOF
   path : IDENT
   line : IMPORT path+ | OPTION IDENT IDENT | declarations
```

Imports. The import construction supports file folder structure (without file extensions) by using reserved symbol / for hierarchy walking.

Options. Each option holds bool value. Language supports following options: 1) girard (enables U : U); 2) pre-eval (normalization cache); 3) impredicative (infinite hierarchy with impredicativity rule); In Anders you can enable or disable language core types, adjust syntaxes or tune inner variables of the type checker.

Declarations. Language supports following top level declarations: 1) axiom (non-computable declaration that breakes normalization); 2) postulate (alternative or inverted axiom that can preserve consistency); 3) definition (almost any explicit term or type in type theory); 4) lemma (helper in big game); 5) theorem (something valuable or complex enough).

```
axiom isProp (A : U) : U
def isSet (A : U) : U := Π (a b : A) (x y : Path A a b), Path (Path A a b) x y
```

Sample declarations. For example, signature isProp (A : U) of type U could be defined as normalization-blocking axiom without proof-term or by providing proof-term as definition.

In this example (A : U), (a b : A) and (x y : Path A a b) are called telescopes. Each telescope consists of a series of lenses or empty. Each lense provides a set of variables of the same type. Telescope defines parameters of a declaration. Types in a telescope, type of a

declaration and a proof-terms are a language expressions exp1.

```
menhir
   ident : IRREF | IDENT
   lense : LPARENS ident+ COLON exp1 RPARENS
   telescope : lense telescope
   params : telescope | []
   declarations :
      | DEF IDENT params DEFEQ exp1
      | DEF IDENT params COLON exp1 DEFEQ exp1
      | AXIOM IDENT params COLON exp1
```

Expressions. All atomic language expressions are grouped by four categories: exp0 (pair constructions), exp1 (non neutral constructions), exp2 (path and pi applcations), exp3 (neutral constructions).

```
menhir
   face : LPARENS IDENT IDENT IDENT RPARENS
   part : face+ ARROW exp1
   exp0 : exp1 COMMA exp0 | exp1
   exp1 : LSQ separated(COMMA, part) RSQ
      | LAM telescope COMMA exp1     | PI telescope COMMA exp1
      | SIGMA telescope COMMA exp1   | LSQ IRREF ARROW exp1 RSQ
      | LT ident+ GT exp1            | exp2 ARROW exp1
      | exp2 PROD exp1               | exp2
```

The LR parsers demand to define exp1 as expressions that cannot be used (without a parens enclosure) as a right part of left-associative application for both Path and Pi lambdas. Universe indicies U_j (inner fibrant), V_k (outer pretypes) and S (outer strict omega) are using unicode subscript letters that are already processed in lexer.

```
menhir
   exp2 : exp2 exp3 | exp2 APPFORMULA exp3 | exp3
   exp3 : LPARENS exp0 RPARENS LSQ exp0 MAP exp0 RSQ
      | HOLE            | PRE           | KAN            | IDJ exp3
      | exp3 FST        | exp3 SND      | NEGATE exp3    | INC exp3
      | exp3 AND exp3   | exp3 OR exp3  | ID exp3        | REF exp3
      | OUC exp3        | PATHP exp3    | PARTIAL exp3   | IDENT
      | IDENT LSQ exp0 MAP exp0 RSQ                      | HCOMP exp3
      | LPARENS exp0 RPARENS                             | TRANSP exp3 exp3
```

3 Semantics

The idea is to have a unified layered type checker, so you can disbale/enable any MLTT-style inference, assign types to universes and enable/disable hierachies. This will be done by providing linking API for pluggable presheaf modules. We selected 5 levels of type checker awareness from universes and pure type systems up to synthetic language of homotopy type theory. Each layer corresponds to its presheaves with separate configuration for universe hierarchies. We want to mention here with homage to its authors all categorical models of

```
def   lang :U := inductive
  {   UNI: cosmos → lang
  |   PI: pure lang → lang
  |   SIGMA: total lang → lang
  |   ID: strict lang → lang
  |   PATH: homotopy lang → lang
  |   GLUE: glue lang → lang
  |   INDUCTIVE: w012 lang → lang
  }
```

dependent type theory: Comprehension Categories (Grothendieck, Jacobs), LCCC (Seely), D-Categories and CwA (Cartmell), CwF (Dybjer), C-Systems (Voevodsky), Natural Models (Awodey). While we can build some transports between them, we leave this excercise for our mathematical components library. We will use here the Coquand's notation for Presheaf Type Theories in terms of restriction maps.

3.1 Universe Hierarchies

Language supports Agda-style hierarchy of universes: prop, fibrant (U), interval pretypes (V) and strict omega with explicit level manipulation. All universes are bounded with preorder

$$Fibrant_j \prec Pretypes_k \tag{1}$$

in which j, k are bounded with equation:

$$j < k. \tag{2}$$

Large elimination to upper universes is prohibited. This is extendable to Agda model:

```
def   cosmos : U := inductive
  {   fibrant: nat
  |   pretypes: nat
  }
```

The **anders** model contains only fibrant U_j and pretypes V_k universe hierarchies.

3.2 Dependent Types

▶ **Definition 1** (Type). *A type is interpreted as a presheaf A, a family of sets A_I with restriction maps $u \mapsto u\ f, A_I \to A_J$ for $f : J \to I$. A dependent type B on A is interpreted by a presheaf on category of elements of A: the objects are pairs (I, u) with $u : A_I$ and morphisms $f : (J, v) \to (I, u)$ are maps $f : J \to$ such that $v = u\ f$. A dependent type B is thus given by a family of sets $B(I, u)$ and restriction maps $B(I, u) \to B(J, u\ f)$.*

We think of A as a type and B as a family of presheves $B(x)$ varying $x : A$. The operation $\Pi(x : A)B(x)$ generalizes the semantics of implication in a Kripke model.

▶ **Definition 2** (Pi). *An element $w : [\Pi(x : A)B(x)](I)$ is a family of functions $w_f : \Pi(u : A(J))B(J, u)$ for $f : J \to I$ such that $(w_f u)g = w_{f\ g}(u\ g)$ when $u : A(J)$ and $g : K \to J$.*

```
def  pure (lang : U) : U := inductive
  {  pi: name → nat → lang → lang → pure lang
  |  lambda: name → nat → lang → lang
  |  app: lang → lang
  }
```

▶ **Definition 3** (Sigma). *The set $\Sigma(x : A)B(x)$ is the set of pairs (u, v) when $u : A(I), v : B(I, u)$ and restriction map $(u, v)\ f = (u\ f, v\ f)$.*

```
def  total (lang : U) : U := inductive
  {  sigma: name → lang → total lang
  |  pair: lang → lang
  |  fst: lang
  |  snd: lang
  }
```

The presheaf with only Pi and Sigma is called **MLTT-72** [4]. Its internalization in **anders** is as follows:

```
def  MLTT-72 (A : U) (B : A → U) : U := Σ
     (Π-form₁ : U)
     (Π-ctor₁ : Pi A B → Pi A B)
     (Π-elim₁ : Pi A B → Pi A B)
     (Π-comp₁ : (a : A) (f : Pi A B), Π-elim₁ (Π-ctor₁ f) a = f a)
     (Π-comp₂ : (a : A) (f : Pi A B), f = λ (x : A), f x)
     (Σ-form₁ : U)
     (Σ-ctor₁ : Π (a : A) (b : B a) , Sigma A B)
     (Σ-elim₁ : Π (p : Sigma A B), A)
     (Σ-elim₂ : Π (p : Sigma A B), B (pr₁ A B p))
     (Σ-comp₁ : Π (a : A) (b: B a), a = Σ-elim₁ (Σ-ctor₁ a b))
     (Σ-comp₂ : Π (a : A) (b: B a), b = Σ-elim₂ (a, b))
     (Σ-comp₃ : Π (p : Sigma A B), p = (pr₁ A B p, pr₂ A B p)), 1
```

3.3 Path Equality

The fundamental development of equality inside MLTT provers led us to the notion of ∞-groupoid as spaces. In this way Path identity type appeared in the core of type checker along with De Morgan algebra on built-in interval type.

```
def   homotopy (lang : U) : U := inductive
  {   PathP: lang → lang → lang
  |   plam: name → lang → lang
  |   papp: lang → lang
  |   I
  |   zero
  |   one
  |   meet: lang → lang
  |   join: lang → lang
  |   neg: lang
  |   system: lang
  |   Partial: lang
  |   transp: lang → lang
  |   hcomp: lang
  |   Sub: lang
  |   inc: lang
  |   ouc: lang
  }
```

▶ **Definition 4** (Cubical Presheaf $\mathbb{I}$)**.** *The identity types modeled with another presheaf, the presheaf on Lawvere category of distributive lattices (theory of De Morgan algebras) denoted with* $\square$ — $\boldsymbol{I} : \square^{op} \to Set$.

▶ **Definition 5** (Properties of **I**)**.** *The presheaf* $\boldsymbol{I}$*: i) has to distinct global elements* 0 *and* 1 *(*B_1*); ii)* $\boldsymbol{I}(I)$ *has a decidable equality for each* I *(*B_2*); iii)* $\boldsymbol{I}$ *is tiny so the path functor* $X \mapsto X^{\boldsymbol{I}}$ *has right adjoint (*B_3*).; iv)* $\boldsymbol{I}$ *has meet and join (connections).*

Interval Pretypes. While having pretypes universe V with interval and associated De Morgan algebra ($\wedge$, $\vee$, -, 0, 1, I) is enough to perform DNF normalization and proving some basic statements about path, including: contractability of singletons, homotopy transport, congruence, functional extensionality; it is not enough for proving β rule for Path type or path composition.

Generalized Transport. Generalized transport transp adresses first problem of deriving the computational β rule for Path types:

```
theorem  Path_β (A : U) (a : A) (C : D A) (d: C a a (refl A a))
     :   Equ (C a a (refl A a)) d (J A a C d a (refl A a))
     :=  λ (A : U), λ (a : A),
         λ (C : Π (x : A), Π (y : A), PathP (<_> A) x y → U),
         λ (d : C a a (<_> a)), <j> transp (<_> C a a (<_> a)) -j d
```

Transport is defined on fibrant types (only) and type checker should cover all the cases Note that transp^i (Path^j A v w) φ u$_0$ case is relying on comp operation which depends on hcomp primitive. Here is given the first part of Simon Huber equations [3] for **transp**:

```
transp^i N φ u₀ = u₀
transp^i U φ A = A
transp^i (Π (x : A), B) φ u₀ v = transp^i B(x/w) φ (u₀ w(i/0))
transp^i (Σ (x : A), B) φ u₀ = (transp^i A φ (u₀.1),transp^i B(x/v) φ (u₀.2))
transp^i (Path^j v w) φ u₀ = <j> comp^i A [ϕ u₀ j, (j=0) ↦ v, (j=1) ↦ w] (u₀ j)
transp^i (Glue [φ ↦ (T,w)] A) ψ u₀ = glue [ϕ(i/1) ↦ t'₁] a'₁ : B(i/1)
```

Partial Elements. In order to explicitly define hcomp we need to specify n-cubes where some faces are missing. Partial primitives isOne, 1=1 and UIP on pretypes are derivable in Anders due to landing strict equality Id in V universe. The idea is that (Partial A r) is the type of cubes in A that are only defined when IsOne r holds. (Partial A r) is a special version of the function space IsOne r → A with a more extensional equality: two of its elements are considered judgmentally equal if they represent the same subcube of A. They are equal whenever they reduce to equal terms for all the possible assignment of variables that make r equal to 1.

```
def Partial' (A : U) (i : I) := Partial A i
def isOne : I -> V := Id I 1
def 1=>1 : isOne 1 := ref 1
def UIP (A : V) (a b : A) (p q : Id A a b) : Id (Id A a b) p q := ref p
```

Cubical Subtypes. For (A : U) (i : I) (Partial A i) we can define subtype A [i ↦ u]. A term of this type is a term of type A that is definitionally equal to u when (IsOne i) is satisfied. We have forth and back fusion rules ouc (inc v) = v and inc (outc v) = v. Moreover, ouc v will reduce to u 1=1 when i=1.

```
def sub' (A : U) (i : I) (u : Partial A i) : V := A [i ↦ u ]
def inc' (A : U) (i : I) (a : A) : A [i ↦ [(i = 1) → a]] := inc A i a
def ouc' (A : U) (i : I) (u : Partial A i) (a : A [i ↦ u]) : A := ouc a
```

Homogeneous Composition. hcomp is the answer to second problem: with hcomp and transp one can express path composition, groupoid, category of groupoids (groupoid interpretation and internalization in type theory). One of the main roles of homogeneous composition is to be a carrier in [higher] inductive type constructors for calculating of homotopy colimits and direct encoding of CW-complexes. Here is given the second part of Simon Huber equations [3] for **hcomp**:

```
hcomp^i N [φ ↦ 0] 0 = 0
hcomp^i N [φ ↦ S u] (S u₀) = S (hcomp^i N [φ ↦ u] u₀)
hcomp^i U [φ ↦ E] A = Glue [φ ↦ (E(i/1), equiv^i E(i/1-i))] A
hcomp^i (Π (x : A), B) [φ ↦ u] u₀ v = hcomp^i B(x/v) [φ ↦ u v] (u₀ v)
hcomp^i (Σ (x : A), B) [φ ↦ u] u₀ = (v(i/1), comp^i B(x/v) [φ ↦ u.2] u₀.2)
hcomp^i (Path^j A v w) [φ ↦ u] u₀ = <j> hcomp^i A[φ ↦ u j, (j=0) ↦ v, (j=1) ↦ w] (u₀ j)
hcomp^i (Glue [φ ↦ (T,w)] A) [ψ ↦ u] u₀ = glue [φ ↦ u(i/1)] (unglue u(i/1))
```

3.4 Strict Equality

To avoid conflicts with path equalities which live in fibrant universes strict equalities live in pretypes universes.

```
def   strict (lang : U) : U := inductive
  {   Id: name → lang
  |   ref: lang → lang
  |   idJ: lang → lang → lang
  }
```

We use strict equality in HTS for pretypes and partial elements which live in V. The presheaf configuration with Pi, Sigma and Id is called **MLTT-75** [5]. The presheaf configuration with Pi, Sigma, Id and Path is called **HTS** (Homotopy Type System).

3.5 Glue Types

The main purpose of Glue types is to construct a cube where some faces have been replaced by equivalent types. This is analogous to how hcomp lets us replace some faces of a cube by composing it with other cubes, but for Glue types you can compose with equivalences instead of paths. This implies the univalence principle and it is what lets us transport along paths built out of equivalences.

```
def   glue (lang : U) : U := inductive
  {   Glue: lang → lang → lang
  |   glue: lang → lang
  |   unglue: lang → lang
  }
```

Basic Fibrational HoTT core by Pelayo, Warren, and Voevodsky (2012).

```
def fiber (A B : U) (f: A → B) (y : B): U := Σ (x : A), Path B (f x) y
def isEquiv (A B : U) (f : A → B) : U := Π (y : B), isContr (fiber A B f y)
def equiv (A B : U) : U := Σ (f : A → B), isEquiv A B f
def contrSingl (A : U) (a b : A) (p : Path A a b)
    : Path (Σ (x : A), Path A a x) (a,<i>a) (b,p) := <i> (p @ i, <j> p @ i ∨ j)
def idIsEquiv (A : U) : isEquiv A A (id A) :=
    λ (a : A), ((a,<i>a), λ (z : fiber A A (id A) a), contrSingl A a z.1 z.2)
def idEquiv (A : U) : equiv A A := (id A, isContrSingl A)
```

The notion of Univalence was discovered by Vladimir Voevodsky as forth and back transport between fibrational equivalence as contractability of fibers and homotopical multi-dimentional heterogeneous path equality. The Equiv → Path type is called Univalence type, where univalence intro is obtained by Glue type and elim (Path → Equiv) is obtained by sigma transport from constant map.

```
def univ-formation (A B : U) := equiv A B → PathP (<i> U) A B
def univ-intro (A B : U) : univ-formation A B := λ (e : equiv A B),
    <i> Glue B (∂ i) [(i = 0) → (A, e), (i = 1) → (B, idEquiv B)]
def univ-elim (A B : U) (p : PathP (<i> U) A B)
  : equiv A B := transp (<i> equiv A (p @ i)) 0 (idEquiv A)
def univ-computation (A B : U) (p : PathP (<i> U) A B)
  : PathP (<i> PathP (<i> U) A B) (univ-intro A B (univ-elim A B p)) p
  := <j i> Glue B (j ∨ ∂ i)
     [(i = 0) → (A, univ-elim A B p), (i = 1) → (B, idEquiv B),
      (j = 1) → (p @ i, univ-elim (p @ i) B (<k> p @ (i ∨ k)))]
```

Similar to Fibrational Equivalence the notion of Retract/Section based Isomorphism could be introduced as forth-back transport between isomorphism and path equality. This notion is somehow cannonical to all cubical systems and is called Unimorphism here.

```
def iso-Form (A B: U) : U₁ := iso A B -> PathP (<i>U) A B
def iso-Intro (A B: U) : iso-Form A B :=
   λ (x : iso A B), isoPath A B x.f x.g x.s x.t
def iso-Elim (A B : U) : PathP (<i> U) A B -> iso A B
 := λ (p : PathP (<i> U) A B),
   ( coerce A B p, coerce B A (<i> p @ -i),
     trans⁻¹-trans A B p, λ (a : A), <k> trans-trans⁻¹ A B p a @-k, ⋆)
```

Orton-Pitts basis for univalence computability (2017):

```
def ua (A B : U) (p : equiv A B) : PathP (<i> U) A B := univ-intro A B p
def ua−β (A B : U) (e : equiv A B) : Path (A → B) (trans A B (ua A B e)) e.1
:= <i> λ (x : A), (idfun=idfun" B @ -i)
    ( (idfun=idfun" B @ -i) ((idfun=idfun' B @ -i) (e.1 x)) )
```

3.6 de Rham Stack

Stack de Rham or Infinitezemal Shape Modality is a basic primitive for proving theorems from synthetic differential geometry. This type-theoretical framework was developed for the first time by Felix Cherubini under the guidance of Urs Schreiber. The Anders prover implements the computational semantics of the de Rham stack.

```
def ι (A : U) (a : A) : ℑ A := ℑ−unit a
def μ (A : U) (a : ℑ (ℑ A)) := ℑ−join a

def is-coreduced (A : U) : U := isEquiv A (ℑ A) (ι A)
def ℑ-coreduced (A : U) : is-coreduced (ℑ A) := isoToEquiv
    (ℑ A) (ℑ (ℑ A)) (ι (ℑ A)) (μ A) (λ (x : ℑ (ℑ A)), <i>x) (λ (y : ℑ A),<i>y)
def ind−ℑβ (A : U) (B : ℑ A → U) (f : Π (a : A), ℑ (B (ι A a))) (a : A)
  : Path (ℑ (B (ι A a))) (ind−ℑ A B f (ι A a)) (f a) := <i> f a
def ind−ℑ−const (A B : U) (b : ℑ B) (x : ℑ A)
  : Path (ℑ B) (ind−ℑ A (λ (i : ℑ A), B) (λ (i : A), b) x) b := <i> b
```

Coreduced induction and its $\beta-$quation.

```
def ℑ−ind (A : U) (B : ℑ A → U) (c : Π (a : ℑ A),
    is-coreduced (B a)) (f : Π (a : A), B (ι A a)) (a : ℑ A) : B a
    := (c a (ind−ℑ A B (λ (x : A), ι (B (ι A x)) (f x)) a)).1.1
def ℑ−indβ (A : U) (B : ℑ A → U) (c : Π (a : ℑ A),
    is-coreduced (B a)) (f : Π (a : A), B (ι A a)) (a : A)
  : Path (B (ι A a)) (f a) ((ℑ−ind A B c f) (ι A a))
 := <i> sec-equiv (B (ι A a)) (ℑ (B (ι A a))) (ι (B (ι A a)), c (ι A a)) (f a) @-i
```

Geometric Modal HoTT Framework: Infinitesimal Proximity, Formal Disk, Formal Disk Bundle, Differential.

```
def ∼ (X : U) (a x' : X) : U := Path (ℑ X) (ι X a) (ι X x')
def 𝔻 (X : U) (a : X) : U := Σ (x' : X), ∼ X a x'
def inf-prox-ap (X Y : U) (f : X → Y) (x x' : X) (p : ∼ X x x')
  : ∼ Y (f x) (f x') := <i> ℑ−app X Y f (p @ i)
def T∞ (A : U) : U := Σ (a : A), 𝔻 A a
def inf-prox-ap (X Y : U) (f : X → Y) (x x' : X) (p : ∼ X x x')
  : ∼ Y (f x) (f x') := <i> ℑ−app X Y f (p @ i)
def d (X Y : U) (f : X → Y) (x : X) (ε : 𝔻 X x)
  : 𝔻 Y (f x) := (f ε.1, inf-prox-ap X Y f x ε.1 ε.2)
def T∞-map (X Y : U) (f : X → Y) (τ : T∞ X) : T∞ Y := (f τ.1, d X Y f τ.1 τ.2)
```

3.7 Flat Modality

The Flat modality ♭ (also known as the discrete modality) is a core primitive of Anders, providing a way to talk about discrete types. In a cohesive type theory, $\flat A$ is the type A equipped with the discrete topology. The Anders prover implements computational semantics for the Flat modality including its interaction with cubical primitives.

```
def Flat (A : U) := ♭ A
def FlatUnit (A : U) (a : A) := ♭-unit a
def FlatCounit (A : U) (x : ♭ A) := ♭-counit x

def isDiscrete (A : U) : U := isEquiv A (♭ A) (FlatUnit A)
def ♭-η (A : U) (x : ♭ A) : Path (♭ A) x (♭-unit (♭-counit x)) := <p> x
def FlatInd (A : U) (B : ♭ A → U) := ind-♭ A B
```

The Flat modality satisfies several computational reduction rules for transport and homogeneous composition, which ensure that ♭ behaves correctly with respect to the cubical structure of the type theory.

```
transp^i (♭ A) φ (♭-unit a) = ♭-unit (transp^i A φ a)
hcomp^i (♭ A) [φ ↦ ♭-unit u] (♭-unit u₀) = ♭-unit (hcomp^i A [φ ↦ u] u₀)
```

▶ **Definition 6** (Discreteness). *A type A is called discrete if the unit map $\flat$-unit : $A \to \flat A$ is an equivalence. This corresponds to the intuition that a type is discrete if its points have no non-trivial paths between them that aren't captured by the discrete topology.*

3.8 Inductive Types

The foundational inductive types in **Anders** are based on Martin-Löf's intuitionistic type theory [**?**]. Instead of a generic inductive definition scheme, **Anders** provides well-founded trees (W-types) as a primitive construction from which other data types can be derived. Basic data types like List, Fin, and Vec are implemented as W-types in the core library.

Empty Type

The empty type **0** (also written as `empty`) is the inductive type with no constructors. It represents logical falsehood. Its induction principle is the principle of explosion (*ex falso quodlibet*).

```
def Empty : U := 0
def Empty-elim (C : Empty → U) (z : Empty) : C z := ind-empty C z
```

Unit Type

The unit type **1** (also written as `unit`) is the inductive type with a single constructor $\star$. It represents logical truth.

```
def Unit : U := 1
def star : Unit := ⋆

def Unit-ind (C : Unit → U) (u : C star) (z : Unit) : C z := ind-unit C u z
```

Boolean Type

The boolean type **2** (also written as `bool`) is the inductive type with two constructors 0_2 and 1_2 (or `false` and `true`). It is used for case analysis and decision procedures.

```
def Bool : U := 2
def false : Bool := 0₂
def true : Bool := 1₂

def Bool-ind (C : Bool → U) (f : C false) (t : C true) (z : Bool)
  : C z := ind-bool C f t z
```

Natural Numbers

The type of natural numbers $\mathbb{N}$ is a primitive type in **Anders**, providing efficient normalization for arithmetic operations. Its elimination rule is provided by the `ind-Nat` primitive.

```
def Nat : U := Nat
def Zero : Nat := 0
def Succ (n : Nat) : Nat := S n

def Nat-ind (C : Nat → U) (z : C Zero) (s : Π (n : Nat), C n → C (Succ n))
  (n : Nat) : C n := ind-Nat C z s n
```

W-Types

Well-founded trees (W-types) are the building blocks for all other inductive types in **Anders**. A W-type $\mathsf{W}(x : A), B(x)$ is specified by a type A of shapes and a family $B : A \to U$ of positions for each shape.

```
def W (A : U) (B : A → U) : U := W A B
def sup (A : U) (B : A → U) (a : A) (f : B a → W A B)
  : W A B := sup a f

def W-ind (A : U) (B : A → U) (C : W A B → U)
  (g : Π (a : A) (f : B a → W A B), (Π (b : B a), C (f b)) → C (sup A B a f))
  (w : W A B) : C w := ind-W A B C g w
```

3.9 Higher Inductive Types

Higher inductive types (HITs) generalize inductive types by allowing constructors that target path types, enabling the representation of cell complexes and other quotient-like structures. **Anders** provides two primitive HIT constructions: coequalizers and hub-and-spoke discs. These primitives, combined with W-types, are sufficient to represent a wide range of HITs, including spheres S^n, suspensions, and truncations, following the Three-HIT theorem approach [**?**].

Coequalizers

The coequalizer $\mathsf{coequ}(f, g)$ of two parallel maps $f, g : A \to B$ is a higher inductive type that identifies the images of f and g in B. That is, for every $a : A$, we have a path $f(a) = g(a)$ in $\mathsf{coequ}(f, g)$. This allows for computable representations of the circle S^1, the torus, and general colimits.

```
def coeq (A B : U) (f g : A → B) : U := coequ A B f g
def iota (A B : U) (f g : A → B) (x : B) : coequ A B f g := ι₂ A B f g x
def resp (A B : U) (f g : A → B) (x : A)
  : Path (coequ A B f g) (ι₂ A B f g (f x)) (ι₂ A B f g (g x))
 := <i> resp A B f g x @ i

def coequ-ind' (A B : U) (f g : A → B) (X : coequ A B f g → U)
  (i : Π (b : B), X (iota A B f g b))
  (ρ : Π (x : A), PathP (<i> X (resp A B f g x @ i)) (i (f x)) (i (g x)))
  (z : coequ A B f g) : X z
 := coequ-ind A B f g X i ρ z
```

Hub-and-Spoke Discs

The hub-and-spoke construction $\mathsf{disc}(S, A)$ attaches a "disc" to a type A for every map $f : S \to \mathsf{disc}(S, A)$. This is achieved by adding a "hub" point for each such f and a "spoke" path connecting the hub to $f(s)$ for every $s : S$. This construction is particularly useful for defining n-truncations and other cell complexes where a point must be connected to an entire family of points.

```
def hs (S A : U) : U := disc S A
def center (S A : U) (a : A) : hs S A := base S A a
def hub' (S A : U) (f : S → hs S A) : hs S A := hub S A f
def spoke' (S A : U) (f : S → hs S A) (s : S)
  : PathP (<i> hs S A) (hub' S A f) (f s) := spoke S A f s

def hs-ind (S A : U) (X : hs S A → U)
  (nCenter : Π (a : A), X (center S A a))
  (nHub : Π (f : S → hs S A) (nF : Π (s : S), X (f s)), X (hub' S A f))
  (nSpoke : Π (f : S → hs S A) (nF : Π (s : S), X (f s)) (s : S),
    PathP (<i> X (spoke' S A f s @ i)) (nHub f nF) (nF s))
  (z : hs S A) : X z := disc-ind S A X nCenter nHub nSpoke z
```

4 Properties

Soundness and completeness link syntax to semantics. Canonicity, normalization, and totality ensure computational adequacy. Consistency and decidability guarantee logical and practical usability. Conservativity and initiality support extensibility and universality.

4.1 Soundness and Completeness

Soundness is proven via cubical sets [11, 12, 13].

4.2 Canonicity, Normalization and Totality

Canonicity and normalization hold constructively [14, 15].

4.3 Consistency and Decidability

Consistency follows from the model [16]. Decidability is achieved for type checking [13].

4.4 Conservativity and Initiality

Conservativity and initiality is discussed bu Shulman[18, 17]. Initiality is implicit in the syntactic construction [12].

5 Conclusion

This paper presents Anders, a proof assistant that reimplements cubicaltt within a Modal Homotopy Type System framework, based on MLTT-80 and CCHM/CHM. It integrates HTS strict equality, infinitesimal modalities, and primitives like suspensions or quotients, with the extension adding simplicial types and Hopf fibrations. Anders offers an efficient, idiomatic system — compiling in under one second — using a syntax of Lean and semantics of cubicaltt and Cubical Agda. As a practical refinement of cubicaltt, Anders serves as an accessible tool for homotopy type theory, with potential for incremental enhancements like a tactic language.

A Annex A. Simon Huber Canonical Equations

```
transp^i N φ u₀ = u₀
transp^i U φ A = A
transp^i (Π (x : A), B) φ u₀ v = transp^i B(x/w) φ (u₀ w(i/0)), w = tFill^-i A φ v, v : A(i/1)
transp^i (Σ (x : A), B) φ u₀ = (transp^i A φ (u₀.1), transp^i B(x/v) φ(u₀.2)), v = tFill^i A φ u₀.1
transp^i (Path^j A v w) φ u₀ = ⟨j⟩
 comp^i A [φ ↦ u₀ j, (j=0) ↦ v, (j=1) ↦ w] (u₀ j)
transp^i G ψ u₀ = glue [φ(i/1) ↦ t′₁] a′₁ : G(i/1),
 G = Glue [φ ↦ (T,w)] A
transp^i (W (x : A), B) φ (sup a f) =
 sup (transp^i A φ a) (transp^i (B(v) → W) φ f), v = tFill^i A φ a
transp^i (Coequ A B f g) φ (ι₂ b) = ι₂ (transp^i B φ b)
transp^i (Coequ A B f g) φ (resp a j) = resp (transp^i A φ a) j
transp^i (Disc S A) φ (base a) = base (transp^i A φ a)
transp^i (Disc S A) φ (hub f) = hub (transp^i (S → Disc S A) φ f)
transp^i (Disc S A) φ (spoke f y j) = spoke (transp^i (S → Disc S A) φ f) (transp^i S φ y) j
transp^i (ℑ A) φ (ℑ-unit a) = ℑ-unit (transp^i A φ a)
transp^i (♭ A) φ (♭-unit a) = ♭-unit (transp^i A φ a)
transp^-i A φ u = (transp^i A(i/1−i) φ u)(i/1−i) : A(i/0)
tFill^i A φ u₀ = transp^j A(i/i∧j) (φ∨(i=0)) u₀ : A

hcomp^i N [φ ↦ 0] 0 = 0
hcomp^i N [φ ↦ S u] (S u₀) = S (hcomp^i N [φ ↦ u] u₀)
hcomp^i U [φ ↦ E] A = Glue [φ ↦ (E(i/1), equiv^i E(i/1−i))] A
hcomp^i (Π (x : A), B) [φ ↦ u] u₀ v = hcomp^i B(x/v) [φ ↦ u v] (u₀ v)
hcomp^i (Σ (x : A), B) [φ ↦ u] u₀ = (v(i/1), comp^i B(x/v) [φ ↦ u.2] u₀.2),
 v = hFill^i A [φ ↦ u.1] u₀.1
hcomp^i (Path^j A v w) [φ ↦ u] u₀ = ⟨j⟩
 hcomp^i A [ φ ↦ u j, (j = 0) ↦ v, (j = 1) ↦ w ] (u₀ j)
hcomp^i G [ψ ↦ u] u₀ = glue [φ ↦ t₁] a₁ : G, G = Glue [φ ↦ (T,w)] A,
t₁ = u(i/1) : T, a₁ = unglue u(i/1) : A, glue [φ ↦ t₁] a1 = t₁ : T
hcomp^i (W (x : A), B) [φ ↦ sup a f] (sup a₀ f₀)
 = sup (hcomp^i A [φ ↦ a] a₀) (hcomp^i (B(v) → W) [φ ↦ f] f₀), v = hFill^i A [φ ↦ a] a₀
hcomp^i (Coequ A B f g) [φ ↦ ι₂ u] (ι₂ u₀) = ι₂ (hcomp^i B [φ ↦ u] u₀)
hcomp^i (Coequ A B f g) [φ ↦ resp a j] (resp a₀ j) = resp (hcomp^i A [φ ↦ a] a₀) j
hcomp^i (Disc S A) [φ ↦ base a] (base a₀) = base (hcomp^i A [φ ↦ a] a₀)
hcomp^i (Disc S A) [φ ↦ hub f] (hub f₀) = hub (hcomp^i (S → Disc S A) [φ ↦ f] f₀)
hcomp^i (Disc S A) [φ ↦ spoke f y j] (spoke f₀ y₀ j)
 = spoke (hcomp^i (S → Disc S A) [φ ↦ f] f₀) (hcomp^i S [φ ↦ y] y₀) j
hcomp^i (ℑ A) [φ ↦ ℑ-unit u] (ℑ-unit u₀) = ℑ-unit (hcomp^i A [φ ↦ u] u₀)
hcomp^i (♭ A) [φ ↦ ♭-unit u] (♭-unit u₀) = ♭-unit (hcomp^i A [φ ↦ u] u₀)
hFill^i A [φ ↦ u] u₀ = hcomp^j A [φ ↦ u(i/i∧j), (i=0) ↦ u₀] u₀ : A
```

B Annex B. HoTT Mathematics

Mathematics in Homotopy Type Theory has seen substantial formalization efforts across multiple proof assistants (including Anders, Cubical Agda, Lean, Coq, etc.). Below is a (non-exhaustive) list of notable results that have been mechanized.

B.1 Formalized Results

- Symmetry: A book project about group theory from the univalent point of view [20].
- Modalities and reflective subuniverses [35].
- Higher Groups (k-symmetric n-groups) [22].
- $\|\Sigma A\|_2$ is a 1-type for all sets A [32].
- Localization and L-separated types [27].
- The James construction and $\pi_4(S^3)$ [21].
- The Cayley–Dickson construction and the H-space structure on S^3 [24].
- The join construction [34].
- Cellular cohomology [23].
- Real projective spaces $\mathbb{R}P^n$ [26].
- Homology theory [31].
- Blakers–Massey connectivity theorem [29].
- Seifert–van Kampen theorem [30].
- Nilpotent types and fracture squares [36].
- Eilenberg–MacLane spaces $K(G, n)$ [33].
- Spectral sequences [28].
- Long exact sequence of homotopy n-groups [25].
- Brouwer's fixed-point theorem in real-cohesion [37].
- Cartan Geometry using a modality [38].
- Synthetic Cohomology Theory in Cubical Agda [39].
- Computing Cohomology Rings in Cubical Agda [40].

B.2 Open Problems

Some notable open problems and challenges in the formalization of homotopy theory include:

- Do more "ordinary" ($\leq$ 2-truncated) mathematics: Is univalence a practical advantage for formalizing scheme or manifold theory?
- Prove Freyd's Adjoint Functor Theorem constructively.
- Calculate more homotopy groups of spheres.
- Is ΣA a 1-type for all sets A?
- Prove that all spheres S^n are ∞-truncated.
- Define the type BSU(2) (a delooping of S^3); then define BSU(n), BSO(n), etc.
- Define the type of U-small semi-simplicial types.
- For which 0- or 1-categories D can we define the type of diagrams Fun(D, U)?
- Define the type of $(\infty, 1)$-categories and develop $(\infty, 1)$-category theory.
- Define the localization type of a 1-category C: the universal type with a functor from C.
- Give a general theory of ∞-quotients.
- Formalize the meta-theory of type theory in itself.

B.3 Discussion

- What is a good foundational framework for mathematics? Which criteria are essential, and which are “nice to have”?
- How does set theory fare according to these criteria?
- How does (homotopy) type theory fare?

Anders aims to provide an efficient and educational environment for exploring and extending these formalizations, particularly through its modal and infinitesimal primitives that support synthetic differential geometry and higher categorical constructions.

References

1 Felix Cherubini. Cartan Geometry in Modal Homotopy Type Theory. https://arxiv.org/pdf/1806.05966.pdf, 2019.

2 Thierry Coquand, Simon Huber, and Anders Mörtberg. On Higher Inductive Types in Cubical Type Theory. https://staff.math.su.se/anders.mortberg/papers/cubicalhits.pdf, 2017.

3 Simon Huber. On Higher Inductive Types in Cubical Type Theory. http://www.cse.chalmers.se/~simonhu/misc/hcomp.pdf, 2017.

4 Martin-Löf. An Intuitionistic Theory of Types, 1972.

5 Martin-Löf. An Intuitionistic Theory of Types: Predicative Part, 1975.

6 Martin-Löf. Intuitionistic Type Theory. https://raw.githubusercontent.com/michaelt/martin-lof/master/pdfs/Bibliopolis-Book-retypeset-1984.pdf, 1980.

7 Christine Paulin-Mohring. Introduction to the Calculus of Inductive Constructions. https://hal.inria.fr/hal-01094195/document, 2015.

8 Vladimir Voevodsky. A simple type system with two identity types. https://www.math.ias.edu/vladimir/sites/math.ias.edu.vladimir/files/HTS.pdf, 2013.

9 Danil Annenkov, Paolo Capriotti, Nicolai Kraus, and Christian Sattler. Two-Level Type Theory and Applications. https://arxiv.org/pdf/1705.03307.pdf, 2019.

10 Andrea Vezzosi, Anders Mörtberg, and Andreas Abel. Cubical Agda: A Dependently Typed Programming Language with Univalence and Higher Inductive Types. https://staff.math.su.se/anders.mortberg/papers/cubicalagda.pdf, 2019.

11 Cyril Cohen, Thierry Coquand, Simon Huber, and Anders Mörtberg. Cubical Type Theory: A Constructive Interpretation of the Univalence Axiom. https://staff.math.su.se/anders.mortberg/papers/cubicalagda.pdf, 2018.

12 Steve Awodey. Type Theory and Homotopy. In *Epistemology versus Ontology: Essays on the Philosophy and Foundations of Mathematics in Honour of Per Martin-Löf*, pages 183–201. Springer, 2012.

13 Thierry Coquand. A Survey of Constructive Models of Univalence. Preprint or Lecture Notes, 2018.

14 Simon Huber. Canonicity for Cubical Type Theory. *Journal of Automated Reasoning*, 61(1-4):173–205, 2017.

15 Thomas Streicher. *Semantics of Type Theory: Correctness, Completeness and Independence Results*. Birkhäuser, Basel, 1991.

16 Marc Bezem, Thierry Coquand, and Simon Huber. A Model of Type Theory in Cubical Sets. Preprint, arXiv:1406.1731, 2014.

17 Michael Shulman. Univalence for Inverse Diagrams and Homotopy Canonicity. *Mathematical Structures in Computer Science*, 25(5):1203–1277, 2015.

18 Martin Hofmann. Syntax and Semantics of Dependent Types. In *Semantics and Logics of Computation*, pages 79–130. Cambridge University Press, 1997.

19 Andrej Bauer and Niels van der Weide. The Three-HITs Theorem. DOI:10.4230/LIPIcs.CVIT.2016.23, https://5ht.co/three.pdf.

20 Marc Bezem, Ulrik Buchholtz, Bjorn Dundas, Dan Grayson, et al. Symmetry: A book project about group theory from the univalent point of view. 2020. https://github.com/UniMath/SymmetryBook.

21 Guillaume Brunerie. The James Construction and $\pi_4(S^3)$ in Homotopy Type Theory. *Journal of Automated Reasoning*, 63.2, pp. 255–284, 2019. DOI: https://doi.org/10.1007/s10817-018-9468-2, arXiv:1710.10307.

22 Ulrik Buchholtz, Floris van Doorn, and Egbert Rijke. Higher Groups in Homotopy Type Theory. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS '18)*, pp. 205–214, 2018. DOI: https://doi.org/10.1145/3209108.3209150, arXiv:1802.04315.

23 Ulrik Buchholtz and Favonia. Cellular Cohomology in Homotopy Type Theory. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS '18)*, pp. 521–529, 2018. DOI: `https://doi.org/10.1145/3209108.3209188`, arXiv:1802.02191.

24 Ulrik Buchholtz and Egbert Rijke. The Cayley–Dickson construction in homotopy type theory. *Higher Structures* 2.1, pp. 30–41, 2018. arXiv:1610.01134.

25 Ulrik Buchholtz and Egbert Rijke. The long exact sequence of homotopy n-groups. 2019. arXiv:1912.08696.

26 Ulrik Buchholtz and Egbert Rijke. The real projective spaces in homotopy type theory. In *32nd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, pp. 1–8, 2017. DOI: `https://doi.org/10.1109/LICS.2017.8005146`, arXiv:1704.05770.

27 J. Daniel Christensen, Morgan Opie, Egbert Rijke, and Luis Scoccola. Localization in Homotopy Type Theory. 2018. arXiv:1807.04155.

28 Floris van Doorn, Jeremy Avigad, Steve Awodey, Ulrik Buchholtz, Egbert Rijke, and Michael Shulman. Spectral Sequences in Homotopy Type Theory. 2019. `https://github.com/cmu-phil/Spectral`.

29 Favonia, Eric Finster, Daniel R. Licata, and Peter L. Lumsdaine. A Mechanization of the Blakers–Massey Connectivity Theorem in Homotopy Type Theory. In *31st Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, pp. 1–10, 2016. arXiv:1605.03227.

30 Favonia and Michael Shulman. The Seifert–van Kampen Theorem in Homotopy Type Theory. In *25th EACSL Annual Conference on Computer Science Logic (CSL 2016)*, LIPIcs vol. 62, pp. 22:1–22:16, 2016. DOI: `https://doi.org/10.4230/LIPIcs.CSL.2016.22`.

31 Robert Graham. Synthetic Homology in Homotopy Type Theory. 2017. arXiv:1706.01540.

32 Nicolai Kraus and Thorsten Altenkirch. Free Higher Groups in Homotopy Type Theory. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS '18)*, pp. 599–608, 2018. DOI: `https://doi.org/10.1145/3209108.3209183`, arXiv:1805.02069.

33 Daniel R. Licata and Eric Finster. Eilenberg–MacLane Spaces in Homotopy Type Theory. In *CSL-LICS '14*, 2014. DOI: `https://doi.org/10.1145/2603088.2603153`.

34 Egbert Rijke. The join construction. 2017. arXiv:1701.07538.

35 Egbert Rijke, Michael Shulman, and Bas Spitters. Modalities in homotopy type theory. *Logical Methods in Computer Science* 16.1, 2020. arXiv:1706.07526, `https://lmcs.episciences.org/6015`.

36 Luis Scoccola. Nilpotent Types and Fracture Squares in Homotopy Type Theory. 2019. arXiv:1903.03245.

37 Michael Shulman. Brouwer's fixed-point theorem in real-cohesive homotopy type theory. *Mathematical Structures in Computer Science* 28.6, pp. 856–941, 2018. DOI: `https://doi.org/10.1017/S0960129517000147`, arXiv:1509.07584.

38 Felix Wellen (Cherubini). Cartan Geometry in Modal Homotopy Type Theory. 2018. arXiv:1806.05966.

39 Guillaume Brunerie, Axel Ljungström, and Anders Mörtberg. Synthetic Cohomology Theory in Cubical Agda. `https://staff.math.su.se/anders.mortberg/papers/zcohomology.pdf`.

40 Thomas Lamiaux, Axel Ljungström, and Anders Mörtberg. Computing Cohomology Rings in Cubical Agda. arXiv:2212.04182, `https://hott-uf.github.io/2023/slides/Lamiaux.pdf`.